

Dimensionality reduction 2

Machine learning

Ing. Tomáš Vičar, Ph.D.

Department of Biomedical Engineering,
FEEC, Brno University of Technology

```
graph LR; Root(( )) --> FS[Feature selection – subset of original features]; Root --> FR[Feature reduction (extraction) – create new features (based on original)]; FS --> FM[Filter methods – based on statistical analysis of the data (model independent)]; FS --> WM[Wrapper methods – based on model performance]; FS --> EM[Embedded methods – part of model training]; FR --> L[Linear – PCA, LDA]; FR --> NL[Non-linear – t-SNE, UMAP, Kernel PCA];
```

Feature selection – subset of original features

Filter methods – based on statistical analysis of the data (model independent)

- Mutual information, Correlation, MRMR
- Last lecture!

Wrapper methods – based on model performance

- forward, backward

Embedded methods – part of model training

- LASSO, decision tree/random forest

Feature reduction (extraction) – create new features (based on original)

Linear – PCA, LDA

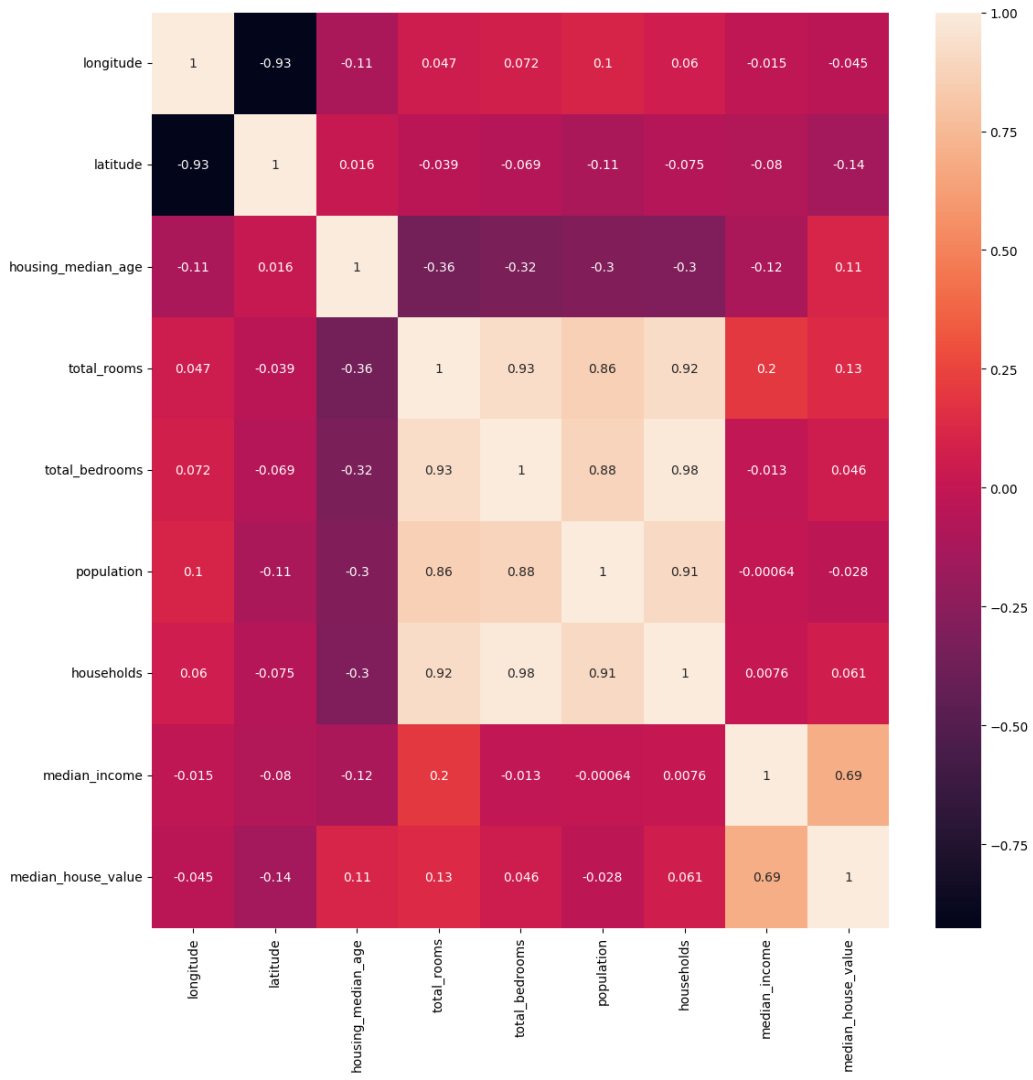
Non-linear – t-SNE, UMAP, Kernel PCA

Feature selection methods

- Keeps the original features – simple interpretation
- Unused features don't have to be measured for new samples.

Feature filtering

- Features are selected based on statistical characteristics of the data.
- These methods are independent of any machine learning algorithm.



Maximum relevance, minimum redundancy (MRMR)

Suppose a feature space F with features x_i ; $i = 1; 2; \dots; P$ and target classes (labels) c . Furthermore suppose its subset $S \subseteq F$. The aim of features selection method is to find S with $M < P$ features x_i ; $i = 1; 2; \dots; M$ according to defined criteria.

Max-Relevance and Min-Redundancy scheme is based on two concepts: a good feature should have **maximum relevance to target class c** and **minimum redundancy to already selected features**.

These two properties can be described by mutual information.

Mutual information

The mutual information I is a measure of the amount of information that one random variable has about another variable:

$$I(x; y) = \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log \left(\frac{p(x(i), y(j))}{p(x(i)) \cdot p(y(j))} \right)$$

where MI is zero when x and y are statistically independent: $p(x(i), y(i)) = p(x) \cdot p(y)$.

The MI can be expressed as:

$$I(x; y) = H(x) + H(y) - H(x, y)$$

Max - Relevance, MRMR - cont'd

The Max-Relevancy scheme selects such features x_i , which have largest mutual information $I(x_i; c)$ with the target class c . The features are selected in such a way that the mean value D of selected mutual information is maximized:

$$\max_S D(S, c); \quad D(S, c) = \frac{1}{P} \sum_{x_i \in S} I(x_i, c)$$

Min-Redundancy - MRMR

The Min-Redundancy scheme selects features according to their redundancies. When two features are highly dependent on each other, their selection does not change so much the respective classification accuracy. Therefore, this scheme for the features selection is based on the minimization of function $R(S)$:

$$\min_S R(S); \quad R(S) = \frac{1}{P} \sum_{x_i, x_j \in S} I(x_i, x_j)$$

MRMR

The MRMR is a combination of these two constraints. The final criteria can be constructed using difference:

$$\max_S (D(S) - R(S))$$

Or ratio:

$$\max_S \frac{D(S)}{R(S)}$$

MRMR

The features selection works in an iterative manner. First, the feature with the highest mutual information to the classes is selected (Max Relevancy). The rest features are selected incrementally, e.g. previously selected features stay in a subset S_{i-1} and the new feature x_i is selected in a such way, that it maximizes the final criterion to create $S_i = S_{i-1} + x_i$.

The basic paper introducing this method:

<https://ieeexplore.ieee.org/document/1453511>

Implemenation and online MRMR:

<http://home.penglab.com/proj/mRMR/>

Feature wrapping

- Set of features is evaluated based on performance of specific model.
- Model is trained on subset of features and evaluated (using validation set or cross-validation).
- Model specific and computationally expensive.
- Leads to good results.

Exhaustive search

- Best what we can do is to try all possible subset of features.
- Number of subsets grows exponentially with number of features.

e.g. 32 subsets for 5 features, 32768 subsets for 15 features

This makes it impractical for large number of features -> heuristic approaches – forward/backward feature search.

Forward selection

The forward selection tries to select a subset of M features (we denote the subset by Z) from whole feature set in an incremental manner.

The *wrapper* first searches for the feature subset with one feature x_1 ($Z_1=\{x_1\}$) that leads to the largest error reduction.

Then, from the set $\{S_n - x_1\}$, the wrapper selects the feature x_2 so that the feature set $Z_2=\{x_1, x_2\}$ leads to the largest error reduction.

This incremental selection repeats until the classification error begins to increase, i.e., $e_{k+1} > e_k$.

Once the termination condition is satisfied, the selected number of features, M , is chosen as the dimension for which the lowest error is first reached.

Backward feature selection

The backward selection tries to exclude one redundant feature at a time from the current feature set S_k (initially, S_k is set to F), with the constraint that the resultant feature set S_{k-1} leads to a classification error e_{k-1} **no worse** than e_k . Because every feature in S_k can be considered in removal, there are k different configurations of S_{k-1} . For each possible configuration, the respective classification error e_{k-1} is calculated. Among the k configurations of S_{k-1} , the one that leads to the largest error reduction is chosen as the new feature set. If, for every configuration, the corresponding error e_{k-1} is larger than e_k , there is no gain in either classification accuracy or feature dimension reduction (i.e. every existing feature in S_k is useful), thus, the backward selection terminates. If there are multiple configurations leading to the same error reduction, one of them is chosen randomly.

Embedded methods

- Feature selection is part of model training.
- The model itself determines which features are important.

Feature selection based on Random Tree

Random forests are among the most popular machine learning methods thanks to their relatively good accuracy, robustness and ease of use. They also provide two straightforward methods for feature selection: mean decrease impurity and mean decrease accuracy.

More details about random trees and forest will be given in a separate lecture.

Feature selection based on LASSO regression

- LASSO regularization can produce zero weights of some features -> they are not used for prediction.
- More details will be provided on next lecture.

Feature reduction methods (PCA, LDA)

Principle Component Analysis (PCA)

Principle Component Analysis (PCA)

PCA is one of the oldest and most widely used dimension reduction method. Its idea is simple - reduce the dimensionality of a dataset, while preserving as much *variability* as possible. It is assumed that the high variability of principal components captures most of the statistical information contained in the dataset.

The input variables are *transformed* into the new/output variables such that they are **linear functions old variables**. During this process the **variance is maximized**. New variables are also **uncorrelated** with each other.

Sources:

https://www.researchgate.net/publication/229158312_Kernel_Principal_Component_Analysis_and_its_Applications_in_FaceRecognition_and_Active_Shape_Models

PCA

Consider a set of points $\mathbf{x}_i$, $i = 1, \dots, n$, in an m -dimensional space such that their mean $\mu = 0$, i.e., centroid is at the origin.

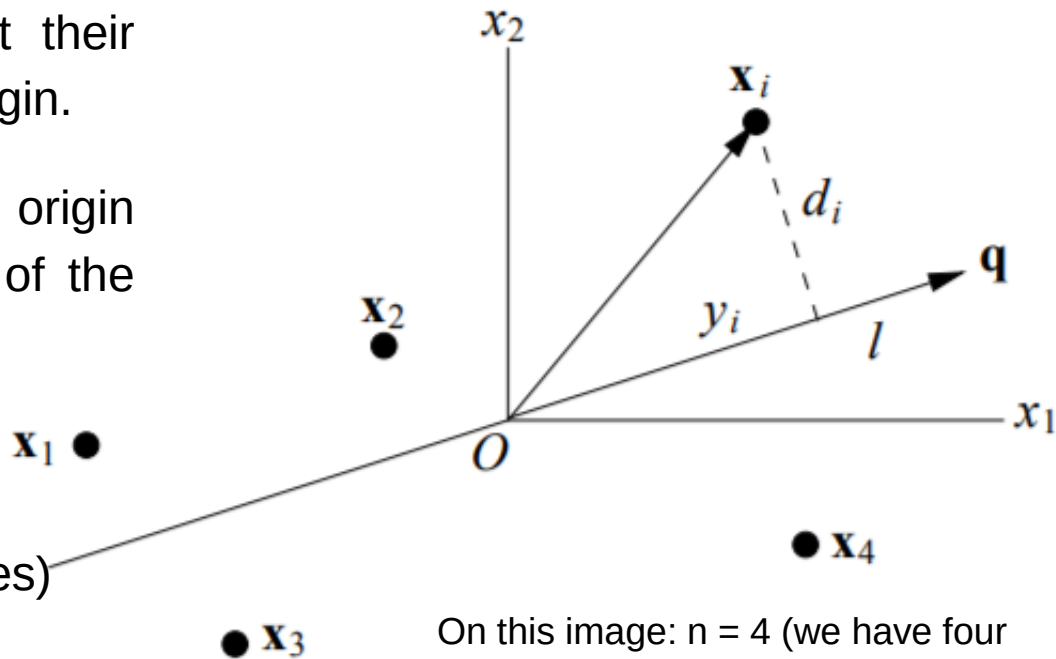
We want to find a line ℓ through the origin that maximizes the projections (y_i) of the points $\mathbf{x}_i$ on ℓ .

Recap:

n - number of points (samples)

m - dimensionality (number of features)

Note: we assume that: $\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{0}$



On this image: $n = 4$ (we have four points) and $m = 3$ (3D space).

Remark: Dot product and vector projection

Consider two vectors (**a**, **b**) and their dot product:

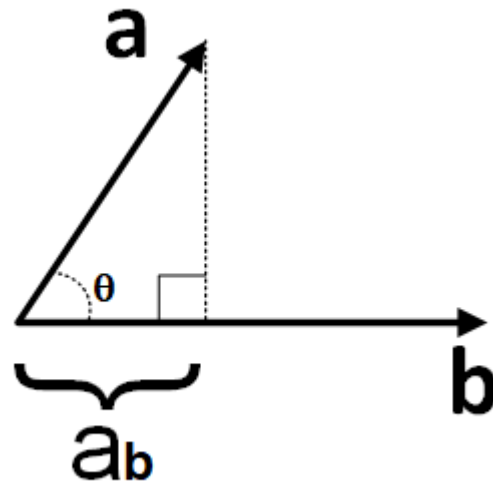
$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

Now consider the (scalar) projection of vector **a** onto **b** vector:

$$a_b = \|\mathbf{a}\| \cos \theta$$

Therefore, the projection and dot product will be the same if we project on unit vector:

$$\|\mathbf{b}\| = 1$$



PCA

Let $\mathbf{q}$ denote the unit vector along line ℓ . Then, the projection y_i (i.e. a **single value** representing the distance from **origin**) of $\mathbf{x}_i$ on ℓ is:

$$y_i = \mathbf{x}_i^\top \mathbf{q}$$

You can check that this projection results in a scalar value for one sample, i.e. vector of feature values. The **mean value of the squared projection**, which is the **variance** V over all samples is:

$$V = \frac{1}{n} \sum_{i=1}^n y_i^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{q})^2 \geq 0.$$

PCA

Because $\mathbf{x}_i^\top \mathbf{q} = \mathbf{q}^\top \mathbf{x}_i$, expanding previous equation gives

$$V = \frac{1}{n} \sum_{i=1}^n (\mathbf{q}^\top \mathbf{x}_i)(\mathbf{x}_i^\top \mathbf{q}) = \mathbf{q}^\top \left[\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right] \mathbf{q}.$$

The middle factor is the **covariance matrix $\mathbf{C}$** of the data points:

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top.$$

[

]

Each element represents average of multiplications between particular features

Now, we want to find a **unit vector $\mathbf{q}$ (i.e. direction of line ℓ)** that maximizes the variance V , i.e.,

$$\text{maximize } V = \mathbf{q}^\top \mathbf{C} \mathbf{q} \quad \text{subject to } \|\mathbf{q}\| = 1.$$

PCA

This is a constrained optimization problem. We will use [Lagrange multiplier method](#):

Let's combine V and the constraint using Lagrange multiplier λ :

$$\text{maximize } V' = \mathbf{q}^\top \mathbf{C} \mathbf{q} - \lambda(\mathbf{q}^\top \mathbf{q} - 1).$$

Now, we differentiate V' with respect to $\mathbf{q}$ and set to 0:

$$\frac{\partial V'}{\partial \mathbf{q}} = 2\mathbf{q}^\top \mathbf{C} - 2\lambda \mathbf{q}^\top = 0.$$

PCA

Rearranging the terms gives:

$$\mathbf{q}^T \mathbf{C} = \lambda \mathbf{q}^T.$$

Since the **covariance matrix $\mathbf{C}$ is symmetric**, $\mathbf{C}^T = \mathbf{C}$. So, transposing both sides of previous equation gives:

$$\mathbf{C} \mathbf{q} = \lambda \mathbf{q}.$$

This equation is called an **eigenvector equation**, where $\mathbf{q}$ is the **eigenvector** (yes, a vector) and λ is the **eigenvalue** (yes, a scalar).

Thus, the eigenvector $\mathbf{q}$ of $\mathbf{C}$ gives the line ℓ that maximizes variance V .

PCA

We showed how to compute the projection only on single line ℓ and we've got a unit eigenvector $\mathbf{q}$. Let's name this (first) eigenvector as $\mathbf{q}_1$ and compute the difference between each feature vector $\mathbf{x}_i$ and projection to $\mathbf{q}_1$:

$$\mathbf{x}'_i = \mathbf{x}_i - \mathbf{x}_i^\top \mathbf{q}_1 \mathbf{q}_1$$

The second term represents the projection (scalar value) of vector $\mathbf{x}_i$ onto the eigenvector ($\mathbf{x}_i^\top \mathbf{q}_1$) and the following multiplication by (unit vector) $\mathbf{q}_1$ gives the vector... so the difference between two vectors can be calculated.

Then - repeat the method on (a new set of features) $\mathbf{x}'_i$ to get $\mathbf{q}_2$, then $\mathbf{q}_3$, $\mathbf{q}_4 \dots$

PCA

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^{\top}.$$

In general, a $m \times m$ covariance matrix $\mathbf{C}$ has m eigenvectors (m is dimensionality):

$$\mathbf{C} \mathbf{q}_j = \lambda_j \mathbf{q}_j, \quad j = 1, \dots, m.$$

Transpose both sides gives:

$$\mathbf{q}_j^{\top} \mathbf{C} = \lambda_j \mathbf{q}_j^{\top}, \quad j = 1, \dots, m,$$

so we get the row vectors $\mathbf{q}_j^{\top}$. Stack it to the matrix gives the final:

$$\begin{bmatrix} \mathbf{q}_1^{\top} \\ \vdots \\ \mathbf{q}_m^{\top} \end{bmatrix} \mathbf{C} = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_m \end{bmatrix} \begin{bmatrix} \mathbf{q}_1^{\top} \\ \vdots \\ \mathbf{q}_m^{\top} \end{bmatrix}$$

PCA

Denote matrix $\mathbf{Q}$ and $\mathbf{\Lambda}$ as

$$\mathbf{Q} = [\mathbf{q}_1 \ \cdots \ \mathbf{q}_m], \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_m).$$

Then the last equation becomes:

$$\mathbf{Q}^\top \mathbf{C} = \mathbf{\Lambda} \mathbf{Q}^\top$$

or

$$\mathbf{C} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top.$$

This is the **matrix equation of eigendecomposition**. Recall, that $\mathbf{C}$ is symmetric covariance matrix.

PCA properties

The eigenvectors are orthonormal:

$$\mathbf{q}_j^\top \mathbf{q}_j = 1,$$

$$\mathbf{q}_j^\top \mathbf{q}_k = 0, \quad \text{for } k \neq j$$

So, the eigenmatrix $\mathbf{Q}$ is orthogonal:

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{Q} \mathbf{Q}^\top = \mathbf{I}$$

The eigenvalues are arranged to be sorted in descending order (representing the described variance) $\lambda_j \geq \lambda_{j+1}$.

Value λ_j is equal to the variance σ_j^2 of data along $\mathbf{q}_j$.

PCA properties

Each entry of particular eigenvector (*loadings*) tells us how much the corresponding feature contribute to the new component.

Projection of each sample onto the eigenvectors provides numbers (*score*), which defines the coordinate of this sample in the new space.

$\mathbf{q}_1$ gives orientation of the largest variance.

$\mathbf{q}_2$ gives orientation of largest variance orthogonal to $\mathbf{q}_1$ (2nd largest variance).

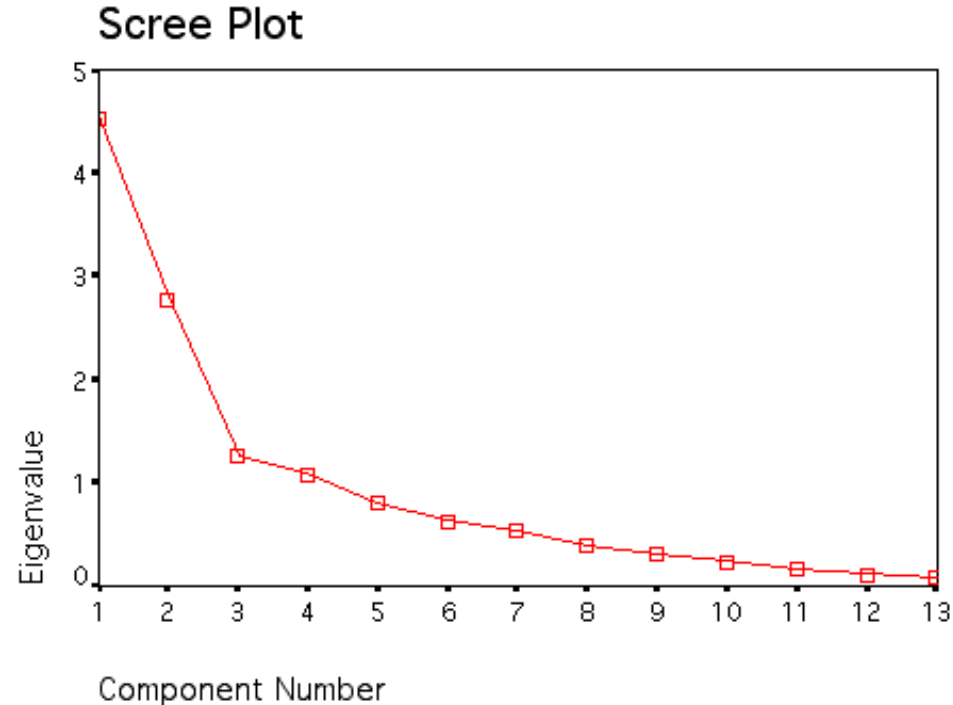
$\mathbf{q}_j$ gives orientation of largest variance orthogonal to $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$ (j-th largest variance). $\mathbf{q}_m$ is orthogonal to all other eigenvectors (least variance)

PCA - Scree plot

A scree plot is a *diagnostic tool* to check whether PCA works well on your data or not. Principal components are created in order of the amount of variation they cover: PC1 captures the most variation, PC2 — the second most, and so on.

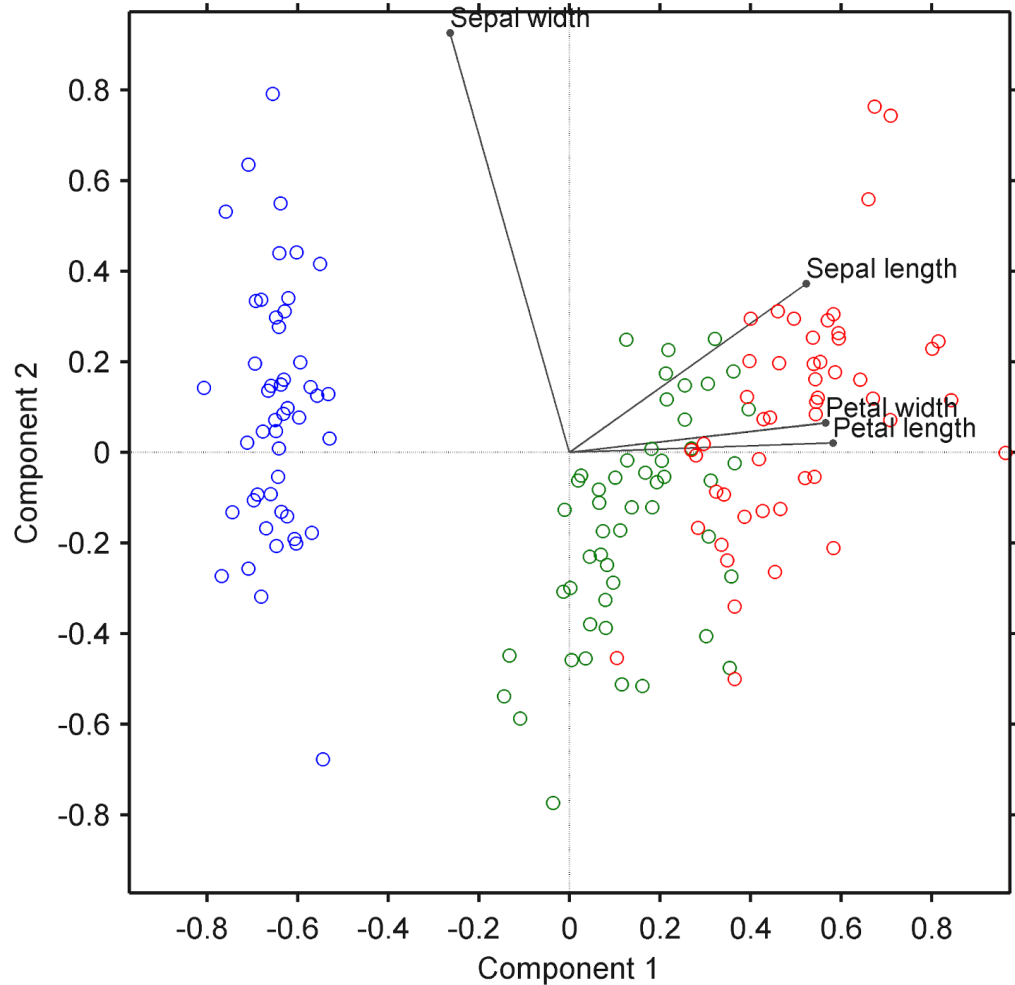
The image shows 13 PCs (there were 13 features). The 1st and 2nd covers most of the variability in the data.

Eigenvalues can be converted to %.



PCA biplot/loading plot

PCA analysis of [Iris flower dataset](#) showing how different features (lines named Sepal width etc.) contribute to 1st and 2nd principal component (axis). Three features on the right contribute almost equally to PC1 while the Sepal width mainly contributes to PC2. The angle between the lines represents “how much the features are correlated”. Angle 90° means “no correlation”. The color dots represent each sample projected into the new (principal component) space. Three colors - three different classes.



PCA limitation

- PCA is a **linear projection method** and it could fail (e.g. lead you to wrong conclusions) on some datasets;
- variance maximization sometimes does not lead to the desired outcome;
- the results of PCA depend on the scaling of features;
- it could be difficult to provide physically meaningful interpretation of generated projections;
- the orthogonality of projection directions can be restrictive as the informative dimensions do not necessarily have to be orthogonal to each other;
- it is potentially computationally demanding to compute (for large number of features);
- it doesn't consider labels.

Linear Discriminant Analysis

LDA

Linear discriminant analysis

In PCA, the class label of each and every example, even if available, is ignored and only the feature values from each example are considered. Thus, PCA is considered an unsupervised approach. The emphasis in PCA is to preserve data variability as much as possible while reducing the dimensionality.

Linear discriminant analysis (LDA) on the other hand makes use of class labels as well and its focus is on finding a lower dimensional space that emphasizes class separability, i.e. it seeks a projection that best separates the data in a least-squares sense (efficient for data discrimination).

References:

<https://iksinc.online/2018/11/12/dimensionality-reduction-via-linear-discriminant-analysis/>

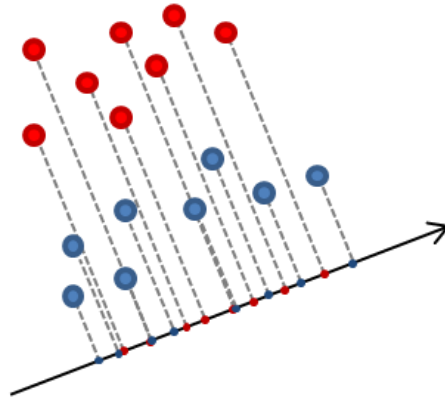
Book: Duda, R. et al. Pattern Classification, Second Edition, John Wiley & Sons, 2001, page 117-121

LDA - principle

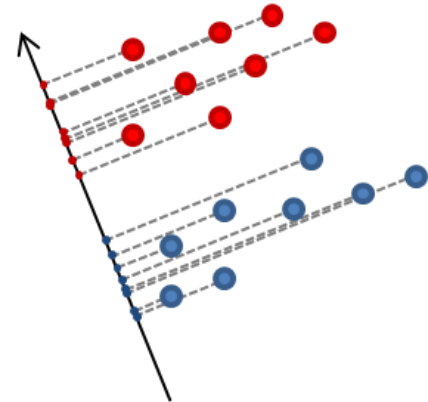
Labelled
data



PCA projection:
Maximising the variance of
the whole set



LDA projection:
Maximising the distance
between groups



LDA

Given that an infinite number of projections are possible, we need to define a measure that can help us to select the best projection. Intuitively, we would like the projected points from two classes to be **as far apart as possible**. We can capture this notion by measuring the **difference between means** of class 1 and class 2 projected points. Although means can be far apart, it is still possible for projected points from two classes to show a fair amount of overlap if the projected points are spread out. To counter this, we also want the projected points from each class to lie **close to each other**. This means the mapped points for each class should have **as small a spread as possible**.

How to find the projection defined by vector $\mathbf{w}$?

LDA

Suppose a two-class problem with labeled training data. There are n_i samples in each class D_i ($i=1,2$). First, we define the *sample* mean feature vector for each class:

$$\mathbf{m}_i = \frac{1}{n_i} \sum_{x \in D_i} \mathbf{x}_i$$

Then the sample mean for projected points (y_i) is given by

$$\begin{aligned} \tilde{m}_i &= \frac{1}{n_i} \sum_{y \in Y_i} y_i = \\ &= \frac{1}{n_i} \sum_{x \in D_i} \mathbf{w}_i^T \mathbf{x} = \mathbf{w}^T \mathbf{m}_i \end{aligned}$$

and it is simply a projection of the mean vector onto $\mathbf{w}$. So we are able to compute the position of means for both classes.

LDA

The purpose of LDA is to **maximize distance** between these projected means:

$$|\tilde{\mathbf{m}}_1 - \tilde{\mathbf{m}}_2| = |\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)|$$

We can make this difference as large as we wish by scaling $\mathbf{w}$. But, we also want to have a **small sample variance** (called here as a *scatter*) within the projected class, i.e.:

$$\tilde{\mathbf{s}}_i^2 = \sum_{y \in Y_i} (\mathbf{y} - \tilde{\mathbf{m}}_i)^2.$$

With the help of this scatter, we define the criterium (**Fisher linear discriminant**), which takes into account distance between projected means and also the total **within class scatter** $\tilde{s}_1^2 + \tilde{s}_2^2$:

$$J(\mathbf{w}) = \frac{|\tilde{\mathbf{m}}_1 - \tilde{\mathbf{m}}_2|^2}{\tilde{\mathbf{s}}_1^2 + \tilde{\mathbf{s}}_2^2}$$

LDA

To find the $\mathbf{w}$, we must rewrite $J(\mathbf{w})$ as an explicit function of $\mathbf{w}$. First, we define the scatter matrices $\mathbf{S}_i$ (each class has one matrix):

$$\mathbf{S}_i = \sum_{x \in D_i} (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T$$

and

$$\mathbf{S}_w = \mathbf{S}_1 + \mathbf{S}_2.$$

Then we can write

$$\begin{aligned} \tilde{s}_i^2 &= \sum_{x \in D_i} (\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \mathbf{m}_i)^2 = \\ &= \sum_{x \in D_i} \mathbf{w}^T (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T \mathbf{w} = \\ &= \mathbf{w}^T \mathbf{S}_i \mathbf{w} \end{aligned}$$

LDA

Therefore the sum of these scatters can be written as:

$$\tilde{s}_1^2 + \tilde{s}_2^2 = \mathbf{w}^T \underline{\mathbf{S}_w} \mathbf{w}$$

Similarly, the separation of projected means:

$$\begin{aligned} (\tilde{m}_1 - \tilde{m}_2)^2 &= (\mathbf{w}^T \mathbf{m}_1 - \mathbf{w}^T \mathbf{m}_2)^2 = \\ &\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w} = \\ &\mathbf{w}^T \underline{\mathbf{S}_B} \mathbf{w}, \end{aligned}$$

where

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$$

LDA

The final expression for J is:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}},$$

where $\mathbf{S}_w$ is called within-class scatter matrix and $\mathbf{S}_B$ is between-class scatter matrix.

Note, that $\mathbf{S}_B$ is computed only from vector of mean values ($(\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$) and $\mathbf{S}_w$ is computed from all features $((\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T)$.

The solution $\mathbf{J}(\mathbf{w})$ can be obtained (without proof) by differentiation with respect to $\mathbf{w}$ and we get:

$$\mathbf{w} = \mathbf{S}_w^{-1} (\mathbf{m}_1 - \mathbf{m}_2)$$

Remark: $J(\mathbf{w})$ is also known as generalized Rayleigh quotient.

LDA

The last term is known as Fisher's linear discriminant (1936), although it is not a discriminant but rather a specific choice of direction for the projection of the data down to one dimension. So we have mapping “many-to-one” dimension.

We found an optimal projection of our data onto 1D space. The last step is missing to obtain the classifier - to find the threshold for projected data. This will be shown later in “Linear classifier” part of this course.

There is also similar explanation for **multiclass** LDA.

LDA and eigenvectors

What can be proofed is a rather general formula:

$$[\mathbf{S}_w^{-1}\mathbf{S}_B]\mathbf{W} = \Lambda\mathbf{W}$$

Actually, this is a definition of eigenvalues and eigenvectors of matrix $\mathbf{X}$:

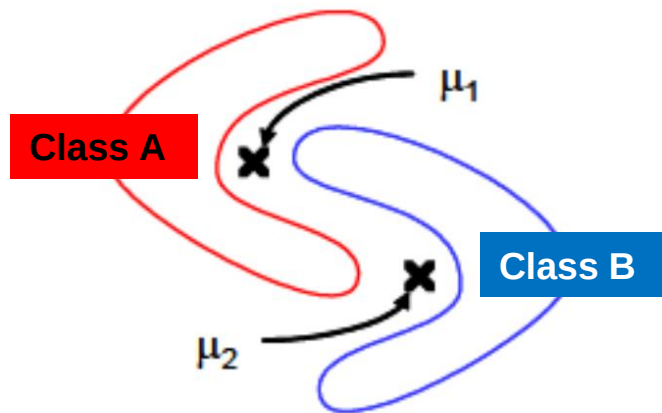
$$\mathbf{XW} = \Lambda\mathbf{W},$$

where Λ is a diagonal matrix of eigenvalues and $\mathbf{W}$ is a matrix of eigenvectors.

Thus computing eigen-decomposition of $[\mathbf{S}_w^{-1}\mathbf{S}_B]$, leads to orthogonal projection to more dimensions according to LDA principle.

Limitations of LDA

- LDA is a *parametric method* since it assumes unimodal Gaussian feature distribution - if the distributions are significantly non-Gaussian, the LDA projections will not be able to preserve any complex structure of the data, which may be needed for classification.



Common nonlinear dimensionality reduction techniques

Sammon mapping (1969)

Curvilinear components analysis (1997)

Kernel PCA (1997)

Isomap (2000)

Locally Linear Embedding (2000)

Laplacian Eigenmaps (2002)

Maximum Variance Unfolding (2004)

t-SNE (2008)

UMAP (2018)

You should be able to

- Justify the need for dimension reduction; differences between feature reduction and feature selection.
- Describe the curse of dimensionality.
- Describe the basic analysis of the features and measures of dependency.
- Discuss the (mutual) histograms and entropies, given the (mutual) histograms.
- Describe the mutual information given the formula and its application in machine learning.
- Summarize principle of PCA; specify main properties and limitations.
- Describe principle of LDA; specify the main differences from PCA.
- Principles and properties of Forward/Backward feature selection.
- Principle of Maximum relevance minimum redundancy.